

ENSF 338: GROUP 3 TERM PROJECT

CAMPUS NAVIGATION & EVENT MANAGEMENT SYSTEM

1. INTRODUCTION

This report documents the design, implementation and analysis of a campus navigation and event management system. The system simulates a university campus environment that helps with finding the shortest walking path between buildings, rooms and event bookings, service request bookings, etc. Our campus map is made up of 15 buildings and 25 edges. Our system is also equipped with a route History and Navigation undo feature that allows users to backtrack/revisit previous routes during a navigation session

2. DESIGN DECISIONS

For the various features, we used a combination of different data structures and algorithms to ensure the program works well. These design decisions were made based on a comparison between various data structures, and we decided to go with what was more efficient based on the complexities

2.1. CAMPUS MAP AND SHORTEST PATH NAVIGATION FEATURE

We decided to present our campus map using a weighted **GRAPH**, and that is because the campus is a network of buildings/locations connected by pathways. We used **Dijkstra's algorithm** for shortest-path navigation because it pairs well with the chosen data structure to find the shortest path between buildings based on the weights of the edges between nodes. Each building/location is represented as a node, while the pathways are represented as weighted edges.

An alternative decision was to use a Breadth-First Search algorithm, but it's not a good choice because it only guarantees the shortest path when all edges have equal weights. Since different buildings have varying distances from each other, a BFS may return a path with steps, but that doesn't guarantee it's the shortest total time possible.

2.2. ROUTE HISTORY AND NAVIGATION UNDO

We decided to implement this feature using a **STACK** because undo operations follow the Last-in, First-out (LIFO) behaviour of a stack. The most recent navigation query goes on top of the stack and is the first one removed (Popped) when the user chooses undo.

An Alternative data structure we considered was a queue, but we realized it is not a good design decision because it has a First-in, First-out behaviour, and the most recent navigation query goes at the bottom of the structure, leading to a linear Time complexity as opposed to a constant Time complexity of a Stack.

2.3. ROOM AND EVENT BOOKING SYSTEM

We decided to implement this feature using a **SORTED LIST** because it is simple to implement and still supports the required operations for this project. Bookings can be kept ordered by date and time, which makes it easier to retrieve events for a day, find events in a time range, and access the next upcoming booking.

An alternative data structure that can be used is an unsorted list. It is not a good choice because every query for upcoming events or time-ordered bookings would require scanning and reorganizing the data repeatedly. This makes retrieval less efficient and less suitable for a system that needs ordered access. A possible better alternative is a BINARY SEARCH TREE because it supports ordered access more efficiently, but a sorted list was chosen to keep implementation simple while still meeting project requirements.

2.4. PRIORITY-BASED SERVICE QUEUE

We decided to implement this feature using a **PRIORITY QUEUE** because service requests must be processed by urgency rather than by arrival time. High-priority requests should be served before low-priority requests, even if they arrive later. This matches the behaviour of a priority queue.

An alternative data structure that can be used is A FIFO QUEUE. A FIFO queue is not a good choice because it processes requests strictly in arrival order. That would mean a low-priority request could be served before a high-priority request, which would not satisfy the feature requirement.

2.5. FAST BUILDING AND RESOURCE LOOKUP

We decided to implement this feature using a **HashMap** because buildings and rooms need to be found quickly by a unique key, such as building ID or room ID. This structure supports very fast insert, delete, and search operations, making it well-suited for direct lookup tasks.

An alternative data structure that can be used is A LIST. A list is not a good choice because searching for a building or room would require checking items one by one, leading to a Linear Time Complexity. As the number of records grows, this becomes slower and does not match the lookup performance requirements.

2.6. INCOMING REQUEST PROCESSING

We decided to implement this feature using a **FIFO QUEUE** because incoming requests must be processed in the exact order they are received. This is a first-in, first-out problem, which is the natural behaviour of a queue.

An alternative data structure that can be used is A STACK. A stack is not a good choice because it would process the most recent request first. That would reverse the arrival order and violate the requirement that requests be handled in the order they were received.

2.7. BALANCED EVENT INDEX

We decided to implement this feature using an **AVL TREE** because it is a self-balancing binary search tree that guarantees efficient insertion and lookup even when data is inserted in sorted order. Since this feature

requires maintaining good performance under adversarial insertion patterns, AN AVL TREE is a strong choice.

An alternative data structure that can be used is A BINARY SEARCH TREE. A BST is not a good choice for the bonus because it can become unbalanced if bookings are inserted in chronological order. In that case, its performance can degrade to linear time.

3. COMPLEXITY ANALYSIS

Operation	Best Case	Average Case	Worst Case	Space	Justification
Add pathway edge	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Appends neighbor entries to adjacency lists
Load map from file	$O(E)$	$O(E)$	$O(E)$	$O(V+E)$	Reads and stores each edge once
Shortest path (Dijkstra)	$O(V^2+E)$	$O(V^2+E)$	$O(V^2+E)$	$O(V)$	Uses linear search for smallest unvisited node
Add route to history	$O(1)$	$O(1)$	$O(n)$	$O(1)$	Append is $O(1)$; removing oldest is $O(n)$
Undo navigation	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Pops from top of stack

Check booking conflict	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Compares against existing bookings
Add booking	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Conflict scan plus sorting
Remove booking	$O(1)$	$O(n)$	$O(n)$	$O(1)$	Linear search by booking ID
Get bookings for day	$O(n)$	$O(n)$	$O(n)$	$O(k)$	Scans all bookings
Get bookings in time range	$O(n)$	$O(n)$	$O(n)$	$O(k)$	Scans and filters overlaps
Get next upcoming booking	$O(1)$	$O(1)$	$O(1)$	$O(1)$	First item in sorted booking list
Add service request	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	Appends then sorts by priority
Serve highest-priority request	$O(n)$	$O(n)$	$O(n)$	$O(1)$	pop(0) shifts elements
Insert building	$O(1)$	$O(1)$	$O(n)$	$O(1)$	Dictionary insertion
Lookup building	$O(1)$	$O(1)$	$O(n)$	$O(1)$	Dictionary lookup

Lookup room	$O(1)$	$O(1)$	$O(n)$	$O(1)$	HashMap lookup
Enqueue incoming request	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Appends to end of list
Dequeue incoming request	$O(n)$	$O(n)$	$O(n)$	$O(1)$	pop(0) shifts elements
AVL insert	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Balance maintained by rotations

4. CHALLENGES AND LEASSON LEARNED

The selection of suitable data structures for each feature, the integration of several subsystems into a single program, the representation of the campus as a weighted graph, the proper handling of file input, and the implementation of booking and queue logic in an accurate and straightforward manner were the primary challenges in this project. Analyzing the time and space complexity of each significant operation instead of concentrating solely on program correctness presented another difficulty.

However, this project demonstrated how choosing the appropriate data structure improves the efficiency and naturalness of issue resolution. Additionally, it demonstrated how applying abstract algorithms like Dijkstra's shortest path to practical issues gives them greater significance. The significance of thorough file organization, organised data representation, full-system testing, and striking a balance between theoretical efficiency and practical simplicity were among the other lessons we learned from this project.

5. DEMO SCENARIOS & SCREENSHOTS

i. Shortest Path Query

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 2
Source building ID: ENG
Destination building ID: LIB
ENG -> MSC -> LIB | Total Time= 5 Minutes
```

***Screenshot showing the Path and Total time it takes to walk from the Engineering Building to the Campus Library**

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 2
Source building ID: ICT
Destination building ID: ADMIN
ICT -> ENG -> ADMIN | Total Time= 7 Minutes
```

***Screenshot showing the Path and Total time it takes to walk from the ICT Building to the Administration Building**

ii. Undo Navigation

=== Campus Navigation and Event Management System ===

1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit

Select an option: 3

Undid route: HNSC -> TFDL

Current location: ADMIN

=== Campus Navigation and Event Management System ===

1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit

Select an option: 3

Undid route: ICT -> ADMIN

Current location: LIB

***Screenshot showing the undo navigation operation and correct state restoration after a sequence of short path queries**

iii. Booking Range Query

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 5
Date (YYYY-MM-DD): 2026-04-20
Start time (HH:MM): 08:00
End time (HH:MM): 19:00
BK001 | ICT-121 | Algorithms Review 1 | 2026-04-20 | 08:00-09:00 | ENSF 338
BK006 | MSC-203 | Research Seminar 6 | 2026-04-20 | 08:00-09:00 | Physics Union
BK002 | ICT-102 | Study Session 2 | 2026-04-20 | 10:00-11:00 | Math Club
BK007 | TFDL-410 | Career Talk 7 | 2026-04-20 | 10:00-11:00 | ENSF 338
BK003 | ENG-201 | Guest Lecture 3 | 2026-04-20 | 12:00-13:00 | Engineering Society
BK008 | TFDL-120 | Club Event 8 | 2026-04-20 | 12:00-13:00 | Math Club
BK004 | ENG-105 | Lab Workshop 4 | 2026-04-20 | 14:00-15:00 | IT Services
BK009 | MTH-211 | Office Hours 9 | 2026-04-20 | 14:00-15:00 | Engineering Society
BK005 | MSC-101 | Project Meeting 5 | 2026-04-20 | 16:00-17:00 | Student Wellness
BK010 | SB-140 | Design Sprint 10 | 2026-04-20 | 16:00-17:00 | IT Services
```

***Screenshot showing the booking within a specified range after a booking range query operation**

iv. Priority Queue Demo

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 8
SR001 | Ava | IT | priority 3

=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 9
Served SR001 (IT)
```

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 8
SR004 | Diego | IT | priority 3

=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Run quick demo
0. Exit
Select an option: 9
Served SR004 (IT)
```

***Screenshots showing service request being read from service_requests.txt file and being served in order of priority**

v. Fast Lookup Demo

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Lookup building by ID
13. Lookup room by ID
14. Add building
15. Add room
16. Delete building
17. Delete room
18. Run quick demo
0. Exit
Select an option: 14
New building ID: KINES
Building name: Kinesiology
Location x: 2.0
Location y: 0.0
Building added.
```

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Lookup building by ID
13. Lookup room by ID
14. Add building
15. Add room
16. Delete building
17. Delete room
18. Run quick demo
0. Exit
Select an option: 12
Building ID: MATH
Building not found.
```

```
=== Campus Navigation and Event Management System ===
1. Show buildings
2. Find shortest path
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Lookup building by ID
13. Lookup room by ID
14. Add building
15. Add room
16. Delete building
17. Delete room
18. Run quick demo
0. Exit
Select an option: 12
Building ID: ENG
Building ID: ENG
Name: Engineering
Location: (1.0, 0.0)
Number of rooms: 2
```

***Screenshots showing Fast Lookup operations including Finding building by ID, adding a new building, lookup for a building that does not exist.**

vi. Request pipeline Demo

```
3. Undo last navigation
4. Show bookings for a day
5. Show bookings in a time range
6. Add booking
7. Remove booking
8. Show next service request
9. Serve next service request
10. Show next incoming request
11. Process next incoming request
12. Lookup building by ID
13. Lookup room by ID
14. Add building
15. Add room
16. Delete building
17. Delete room
18. Run quick demo
0. Exit
Select an option: 18

--- Quick Demo ---
ICT -> ENG -> ADMIN -> HNSC | Total Time= 10 Minutes
Undid route: ICT -> HNSC

Bookings on 2026-04-20:
BK001 | ICT-121 | Algorithms Review 1 | 2026-04-20 | 08:00-09:00 | ENSF 338
BK006 | MSC-203 | Research Seminar 6 | 2026-04-20 | 08:00-09:00 | Physics Union
BK002 | ICT-102 | Study Session 2 | 2026-04-20 | 10:00-11:00 | Math Club
BK007 | TFDL-410 | Career Talk 7 | 2026-04-20 | 10:00-11:00 | ENSF 338
BK003 | ENG-201 | Guest Lecture 3 | 2026-04-20 | 12:00-13:00 | Engineering Society
BK008 | TFDL-120 | Club Event 8 | 2026-04-20 | 12:00-13:00 | Math Club
BK004 | ENG-105 | Lab Workshop 4 | 2026-04-20 | 14:00-15:00 | IT Services
BK009 | MTH-211 | Office Hours 9 | 2026-04-20 | 14:00-15:00 | Engineering Society
BK005 | MSC-101 | Project Meeting 5 | 2026-04-20 | 16:00-17:00 | Student Wellness
BK010 | SB-140 | Design Sprint 10 | 2026-04-20 | 16:00-17:00 | IT Services

Next service request:
SR001 | IT | priority 3

Processing first three incoming requests:
Processed RQ001 | navigation | ICT->ENG
Processed RQ002 | service | Reset password
Processed RQ003 | navigation | ENG->TFDL
```

***Screenshot showing request Pipeline Demo.**

6. GROUP MEMBERS & CONTRIBUTIONS

The table below shows the group members and their contributions to this project.

GROUP MEMBER- UCID	PRIMARY CONTRIBUTION
ADESOPE AJAYI - 30194173	Worked on program text files, booking_system.py , Project Report & Demo video
SHIV SHARMA - 30219086	Worked on models.py and main.py
AMITESH SAINI - 30244391	Worked on navigation.py, graph.py
UTKARSH SUDHIR - 30240320	Worked on hashmap.py & lookup.py
JULIA IZUOGU - 30242363	Worked on Demo.py & HelperFunctions.py
FIEMNA EKPOMBANG - 30253697	Worked on queues.py & HelperFunctions.py

***All group members were involved in the design decisions and complexity analysis of the data structures and algorithms used during this project**